HDOC DAY-55

Sliding Window Maximum Using Pointers in C

Problem Statement

Write a C program that reads the size and elements of a variable-length integer array arr[] and an integer k such that 0 < k <= size. Pass the array to a function named window_maximum() using its address. The function must consider every contiguous subarray of size k as a sliding window, find the maximum element in each window using a nested-loop approach, and display the maxima from left to right separated by spaces. Display -1 if k is invalid.

Algorithm

1. Read the array size n.

2. Read n integer elements into arr.

3. Read the window size k.

4. Call window_maximum(arr, n, k); the array address is passed through a pointer.

5. If k <= 0 or k > n, print -1 and return.

6. For each starting index i from 0 to n-k, assume *(arr+i) is the maximum.

7. Using an inner loop, compare the remaining k-1 elements in that window with the current maximum.

8. Print the maximum of each window, separated by spaces.

C Program

```
#include <stdio.h>

void window_maximum(int *arr, int size, int k)
{
    int i, j, max;

    if (k <= 0 || k > size) {
        printf("-1\n");
        return;
    }

    for (i = 0; i <= size - k; i++) {
        max = *(arr + i);

        for (j = 1; j < k; j++) {
            if (*(arr + i + j) > max)
                max = *(arr + i + j);
        }

        printf("%d", max);
        if (i < size - k)
            printf(" ");
```

```
    }
    printf("\n");
}

int main(void)
{
    int size, k, i;

    printf("Enter size of array: ");
    scanf("%d", &size);

    if (size <= 0) {
        printf("-1\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);

    printf("Enter window size k: ");
    scanf("%d", &k);

    window_maximum(arr, size, k);

    return 0;
}
```

Sample Output

Valid Input

```
Enter size of array: 8
Enter 8 elements: 1 3 -1 -3 5 3 6 7
Enter window size k: 3
3 3 5 5 6 7
```

Invalid k

```
Enter size of array: 5
Enter 5 elements: 2 4 1 7 3
Enter window size k: 6
-1
```

Test Cases

S.No.	Size	Array Elements	k	Expected Output
1	8	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7
2	5	9 8 7 6 5	2	9 8 7 6
3	4	-5 -2 -8 -1	4	-1

4	5	2 4 1 7 3	6	-1
5	3	10 20 30	0	-1

Result

The program successfully finds and displays the maximum element of every contiguous subarray of size k using pointer-based array access and nested loops. It prints -1 when the window size is invalid.